ReSplat: Degradation-agnostic Feed-forward Gaussian Splatting via Self-guided Residual Diffusion

2026. 10. 07

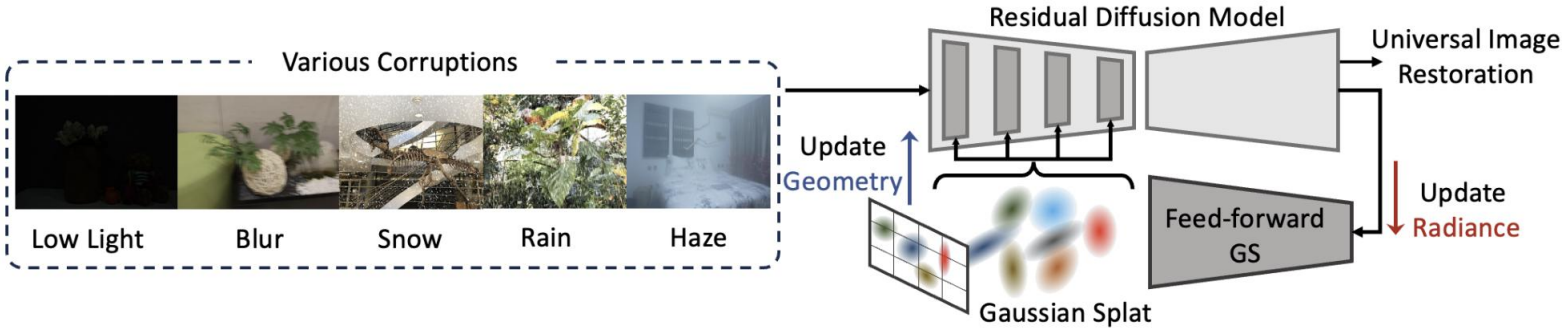
Seonyeong Jeong

Covered Paper

ReSplat: Degradation-agnostic Feed-forward Gaussian Splatting via Self-guided Residual Diffusion (ICLR 2026 Poster)

Youngho Yoon, Kuk-Jin Yoon

KeyWords: 3D Gaussian Splatting(3DGS), Novel View Synthesis(NVS), Image Restoration



Introduction

Problem: **Degraded multi-view inputs에서의 Novel View Synthesis**

기존 연구의 한계

- Generalizable NeRF/3DGS는 주로 clean multi-view images를 전제로 개발됨
- Degradation을 다루는 연구도 특정 corruption 타입에 특화됨
- GAURA는 다양한 degradation을 다루지만, 2D image restoration prior를 활용하지 않음

➡️ 새로운 generalizable gaussian splatting framework, **ReSplat** 제안

Related Works

1. Generalizable Radiance Fields

NeRF 모델이 핵심 → NeRF 기반 다양한 방법들 → Generalizable NeRF
: 이미지의 정보(deep features, depth maps, ...)를 활용해 scene의 radiance field를 예측하고, volume rendering으로 target view 생성

✅ clean image 기반으로 연구된 한계를 residual diffusion model로 해결 목표

2. Novel View Synthesis with Degradations

Deblur-NeRF는 blur kernel, RawNeRF는 raw data, LLNeRF, Aleth-NeRF는 low-light 다룸

DiET-GS, DiSR-NeRF 등은 diffusion prior를 사용하지만 특정 corruption 타입에 맞춰 개발됨

→ 모두 pretrained universal image restoration model 명시적 사용 X, 특정 degradation regime에 초점

✅ 사전학습된 restoration prior를 적극 활용해서 단일 모델로 다양한 degradation 처리 목표

3. Universal Image Restoration (UIR)

Contrastive learning으로 distribution align한 AiRnet, SVD로 degradation type 구분하고 reformulation으로 깨끗한 이미지 recon.하는 IDR

Degradation 관련 feature 포착하는 prompt 사용하는 PromptIR, large-scale model의 prior를 활용하는 DA-CLIP, residual diffusion을 활용한 DiffUIR

Preliminaries: Feed-forward 3DGS

3D Gaussian Splatting

Scene을 anisotropic 3D Gaussians의 집합으로 표현

Gaussian Primitive: center, covariance, opacity, color $(\mu_j, \Sigma_j, \alpha_j, c_j)$

Image plane에 투영한 후 depth 순서로 alpha blending 수행

Gaussian representation & rendering

$$G(x) = e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)}$$

$$C(x') = \sum_{i \in N} c_i \sigma_i \prod_{j=1}^{i-1} (1 - \sigma_j), \quad \sigma_i = \alpha_i G'_i(x')$$

Feed-forward Prediction

Posed multi-view images로부터 각 pixel에 대응하는 Gaussian parameters를 직접 예측

$$\phi : \{(I^i_{\text{in}}, \Pi^i)\}_{i=1}^N \longmapsto \{(\mu_j, \Sigma_j, \alpha_j, c_j)\}_{j=1}^{H \times W \times N}$$

Vanilla 3DGS는 scene별 Gaussian parameters를 최적화하고,
Feed-forward 3DGS는 소수의 posed views로부터 한 번의 forward pass로 이를 예측함

Preliminaries: Residual Diffusion

Residual Denoising Diffusion Model: degradation residual을 예측하여 복원

Residual: degraded image와 clean image의 차이

Forward: residual과 Gaussian noise를 점진적으로 추가함 $q(I_t|I_{t-1}, I_{res}) = \mathcal{N}(I_t; I_{t-1} + \alpha_t I_{res}, \beta_t^2 \mathbf{I})$

Universal RDDM — DiffUIR: Shared Distribution Term (SDT) 추가

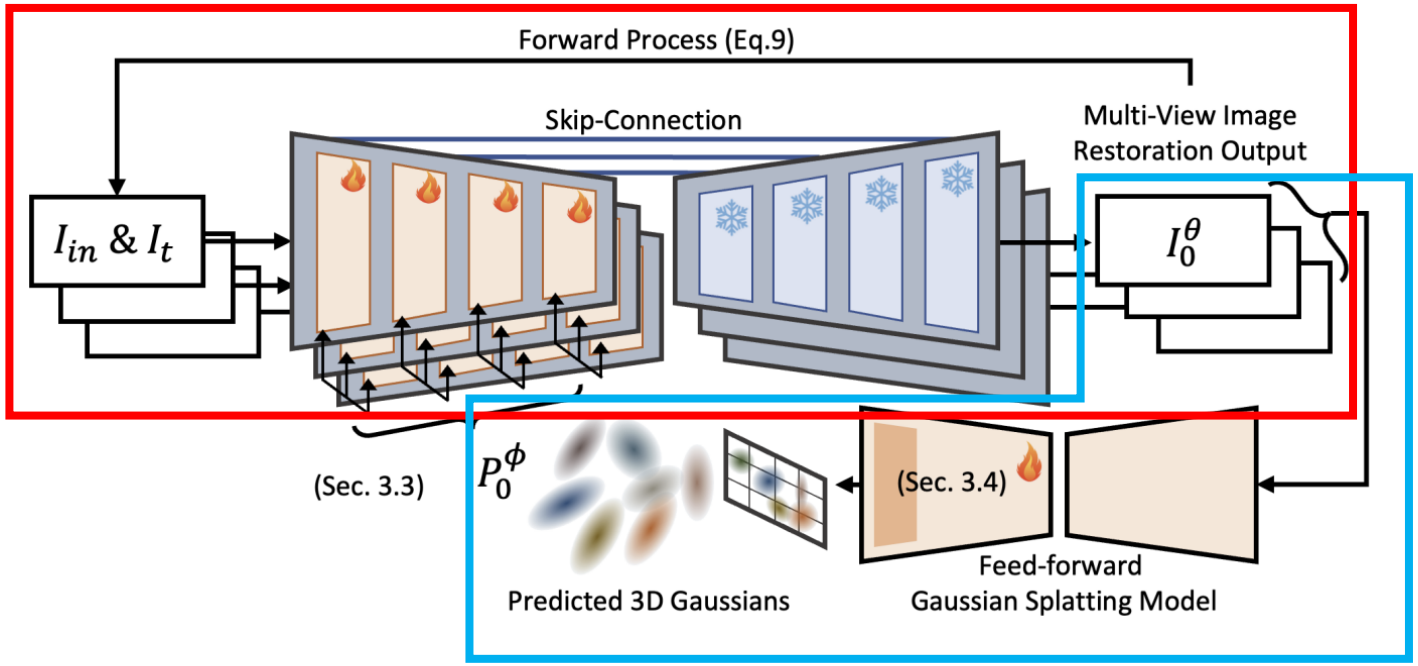
Degraded input 기반 SDT를 추가하여 universal image restoration에 적용 $I_t = I_{t-1} + \alpha_t I_{res} + \beta_t \epsilon_{t-1} - \delta_t I_{in}$

Reverse: 예측 residual, noise를 제거하고 SDT를 반영함 $I_{t-1} = I_t - \alpha_t I_{\text{res}}^{\theta} + \delta_t I_{\text{in}} - (\beta_t^2/\bar{\beta}_t)\epsilon^{\theta}$

Pseudo clean image: degraded input에서 예측 residual을 빼서 얻음 $I_0^{\theta} = I_{in} - I_{res}^{\theta}$

→ ReSplat은 이 중간 복원 결과로 sampling 중 3D geometry를 추정함

Overall Framework



- **Diffusion based Image Restoration model**

  Residual image 반복 추정해서 original image 복원

- **Feed-forward GS model**

  Difffusion에서 중간 복원 image로 gaussian point 정보 예측 → Diffusion에 다시 입력해서 multi-view 정보 제공

Overall Framework: Training stage

Algorithm 1 Training

Input: Clean image, Degraded image: I_0, I_{in};
GT novel view image : I_{nv};
GT residual map: $I_{\text{res}} = I_{\text{in}} - I_0$;
Image Restoration Model: $\theta(*)$;
Feed-forward GS Model: $\phi(*)$;

1: **repeat**
2: $I_0 \sim q(I_0)$;
3: $P_0^{\phi} = \phi(I_0, I_{in})$;
4: $t \sim \text{Uniform}(1, \ldots, T)$;
5: $\epsilon \sim \mathcal{N}(0, I)$;
6: $I_t = I_0 + \bar{\alpha}_t I_{\text{res}} + \bar{\beta}_t \epsilon - \bar{\delta}_t I_{\text{in}}$;
7: Take the gradient descent step on

$$\nabla_\theta \| I_{\text{res}} - I^{\theta}_{\text{res}}(P_0^{\phi}, I_t, I_{\text{in}}, t) \|_1 +$$
$$\nabla_\phi \| I_{nv} - I^{\phi}_{\text{ren}}(I_{\text{in}} - I^{\theta}_{\text{res}}, I_{\text{in}}) \|_1;$$

8: **until** converged

전 페이지에서 봤듯이, feed-forward GS 모델과 Universal Image Restoration 모델을 결합함

→ 상호 보완적인 역할

- feed-forward GS

  point clouds를 명시적으로 추출해야 하므로
  모델 내부에서 필연적으로 Multi-view Stereo를 수행함
  → 3D scene geometry 정보 얻음
  ✅ UIR 모델이 인접한 multi-view images의 대응점을 찾도록 도움

- UIR 모델
  (DiffUIR 채택: 반복적인 scene geomtry extraction & image refinement → 결과 점진적 개선)

  degradation-agnostic image restoration을 수행
  ✅ feed-forward GS가 corruption이 제거된 images로 NVS를 수행하도록 도움

Training Process: Algorithm 1

universal restoration loss L_{UIR}에 해당하며, UIR 모델 θ의 residual prediction을 supervise

novel-view rendering loss L_{NV}를 정의하며, ground-truth clean novel view I_{nv}를 사용하여 feed-forward GS 모델 φ를 supervise함

Overall Framework: Sampling stage

Algorithm 2 Sampling

Require: Degraded image: I_{in};
Image Restoration Model: $\theta(*)$;
Feed-Forward GS Model: $\phi(*)$;

1: $\epsilon \sim \mathcal{N}(0, I)$;
2: $I_T = (1 - \bar{\delta}_T) I_{in} + \bar{\beta}_T \epsilon$;
3: $P_0^{\phi} = None$;
4: **for** $t = T, \ldots, 1$ **do**
5: **if** $t > 1$ **then**
6: $I_{t-1} = I_t - \alpha_t I_{res}^{\theta}(P_0^{\phi}, I_t, I_{in}, t) + \delta_t I_{in}$;
7: $P_0^{\phi} = \phi(I_{\text{in}} - I_{\text{res}}^{\theta}, I_{in})$;
8: **else**
9: $I_{t-1} = I_{in} - I_{res}^{\theta}(P_0^{\phi}, I_t, I_{in}, t)$;
10: $I_{nv} = I_{\text{ren}}^{\phi}(I_{\text{in}} - I_{\text{res}}^{\theta}, I_{\text{in}})$;
11: **return** I_0, I_{nv}

Sampling Process: Algorithm 2

Lines 1–3 | 초기화

Degraded input I_{in}과 Gaussian noise로 I_T 생성

Lines 4–7 | 반복 복원 및 geometry 갱신 $(t > 1)$

Line 6: 현재 geometry $\left(P_0^{\phi}\right)$를 조건으로 residual 예측해서 제거하고 SDT 보정을 통해 I_{t-1} 계산

Line 7: 추정 clean images $I_{in} - I_{res}^{\theta}$로 P_0^{ϕ}를 갱신하여 다음 iteration의 guidanc로 사용

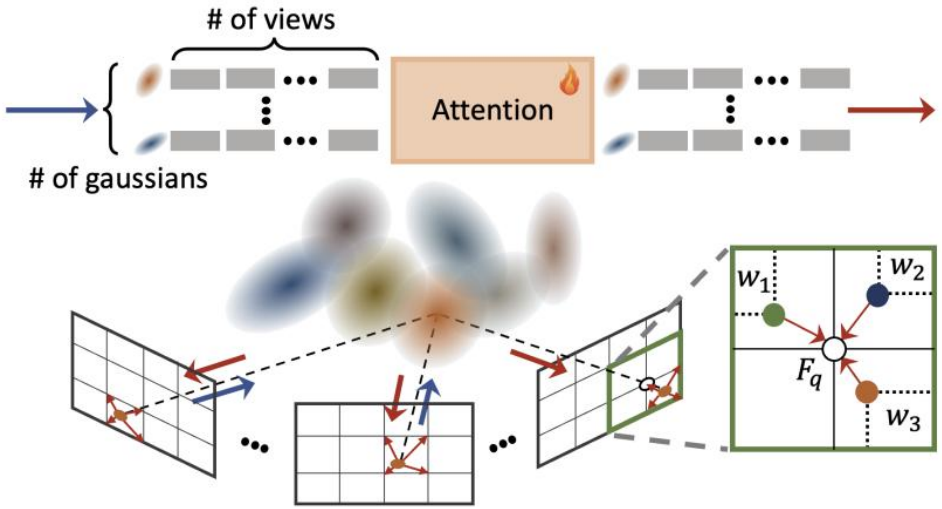
Lines 9–11 | 최종 복원 및 rendering $(t = 1)$

Line 9: $I_{in} - I_{res}^{\theta}$로 최종 복원 images I_0 얻음

Lines 10–11: GS rendering으로 novel-view image I_{nv} 생성 → I_0와 I_{nv} 반환

이때 original corrupted images를 조건으로 하는 feature pre-filtering을 multi-view feature aggregation 이전에 적용하여 잔여 artifacts의 영향을 줄임 (§3.4)

GS Guided Multi-view Alignment

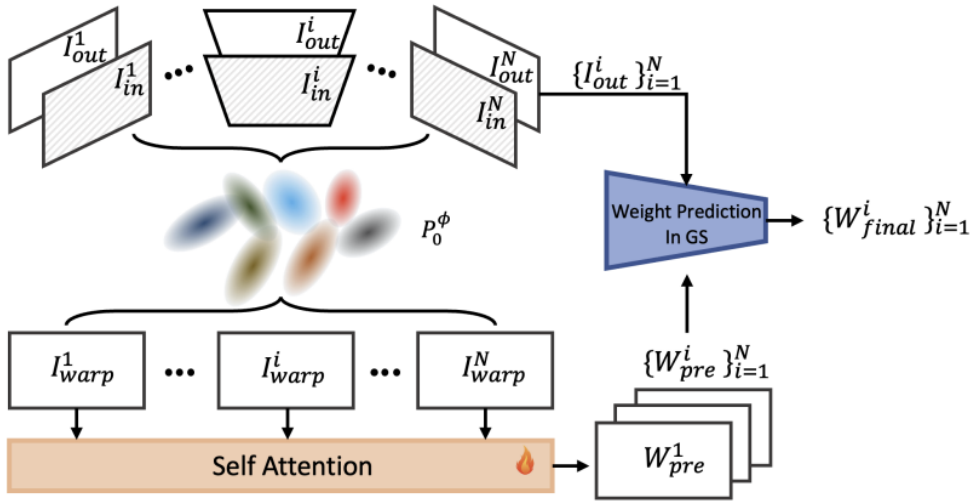


Gaussian centers를 공통 대응점으로 활용

1. 여러 view의 features를 동일한 3D gaussian center에 대응시킴
2. 같은 center의 N개 feature vectors 간 self-attention 수행

   Diffusion 모델의 encoder 내부에서 반복 적용하여
   multi-view restoration의 3D consistency 향상
3. 처리된 feature vector들을 각 view의 원래 pixel 좌표로 reprojection
4. 연속 좌표의 features를 2D interpolation weights로 주변 discrete pixels에 전달

Pre-filtering with Warped Features



Residual artifact의 view별 기여도 조절

1. 복원 images와 degraded inputs를 geometry로 novel pose에 depth-warp함
2. Warped inputs로 view별 reliability map 예측
3. 기존 GS aggregation weight에 reliability를 곱하여 최종 weight 생성

→ Residual artifact가 강하거나 view 간 불일치하는 영역의 기여도를 낮춤

→ soft, degradation-aware gate로 작동함

Experiment Settings

Datasets

- **Training**: IBRNet training dataset에 GAURA의 synthetic degradation pipeline 적용
- Test:
  - **Synthetic**: LLFF dataset에 sythetic degradation 적용
  - **Real-world**: DeblurNeRF (motion blur), REVIDE (haze), LLNeRF (low-light condition), NTURain (rain), RSVD (snow)

Network

- **Image Restoration backbone**: pretrained DiffUIR

  비교 Baselines: AiRnet, PromptIR, DiffUIR는 동일 training data로 fine-tuning, GAURA와 NVS 비교 수행
- **Feed-forward GS backbone**: MVSGaussian

  training process를 빠르게 하기 위해, GS를 restoration 없이 먼저 학습함

Inference에서 DDIM sampling strategy 사용 — 총 3 sampling steps → 3-view 입력을 1초만에 처리

Quantitative Analysis: Synthetic Degradation

Method	Operation	Year	Corruption Type	Novel View Synthesis			Multi-View Image Restoration		
				PSNR(↑)	SSIM(↑)	LPIPS(↓)	PSNR(↑)	SSIM(↑)	LPIPS(↓)
AiRnet	IR → NV	2022	Motion Blur	20.11	0.6896	0.4250	21.99	0.7543	0.3769
PromptIR	IR → NV	2023		20.04	0.6872	0.4208	22.14	0.7526	0.3668
GAURA	Only NV	2024		21.28	0.7198	0.4343	-	-	-
DiffUIR	IR → NV	2024		22.75	0.7824	0.3209	26.34	0.8640	0.2951
ReSplat	IR w/ NV	2025		23.15	0.8049	0.3151	27.14	0.8850	0.2713
AiRnet	IR → NV	2022	Snow	20.22	0.6852	0.3026	21.57	0.8184	0.2159
PromptIR	IR → NV	2023		20.54	0.7067	0.2737	23.21	0.8578	0.1912
GAURA	Only NV	2024		20.48	0.7044	0.3195	-	-	-
DiffUIR	IR → NV	2024		24.24	0.8549	0.1826	31.20	0.9627	0.1019
ReSplat	IR w/ NV	2025		24.46	0.8614	0.1677	32.07	0.9685	0.0886
AiRnet	IR → NV	2022	Haze	9.159	0.3841	0.3892	8.871	0.4155	0.2949
PromptIR	IR → NV	2023		9.784	0.4651	0.3508	9.585	0.5251	0.2280
GAURA	Only NV	2024		17.22	0.7205	0.4516	-	-	-
DiffUIR	IR → NV	2024		21.56	0.8392	0.1857	25.57	0.9612	0.0701
ReSplat	IR w/ NV	2025		21.99	0.8471	0.1750	26.45	0.9680	0.0619
AiRnet	IR → NV	2022	Low-light	9.526	0.1364	0.6041	6.388	0.0859	0.7654
PromptIR	IR → NV	2023		6.367	0.0805	0.6240	6.298	0.0805	0.5789
GAURA	Only NV	2024		15.28	0.6627	0.5177	-	-	-
DiffUIR	IR → NV	2024		18.87	0.8241	0.2429	21.88	0.9374	0.1647
ReSplat	IR w/ NV	2025		19.76	0.8276	0.2433	22.82	0.9452	0.1605
AiRnet	IR → NV	2022	Rain	20.49	0.6988	0.3416	23.09	0.8008	0.3018
PromptIR	IR → NV	2023		20.71	0.7175	0.2992	24.78	0.8555	0.2499
GAURA	Only NV	2024		21.78	0.7578	0.4110	-	-	-
DiffUIR	IR → NV	2024		23.51	0.8313	0.2538	29.69	0.9357	0.1919
ReSplat	IR w/ NV	2025		24.11	0.8505	0.2140	31.28	0.9538	0.1533

NVS의 PSNR과 SSIM, IR의 모든 지표에서 최고 성능

예외: Low-light NVS LPIPS는 DiffUIR 0.2429, ReSplat 0.2433으로 DiffUIR가 근소 우세 (테이블에 표시 잘못됨)

Quantitative Analysis: Mixed Degradation

Method	Corruption Type	Novel View Synthesis			Multi-View Image Restoration		
		PSNR(↑)	SSIM(↑)	LPIPS(↓)	PSNR(↑)	SSIM(↑)	LPIPS(↓)
DiffUIR	Rain+Motion Blur	20.07	0.6910	0.4885	20.41	0.7083	0.4653
ReSplat		20.44	0.7090	0.4555	20.74	0.7220	0.4507
DiffUIR	Snow+Motion Blur	21.63	0.7407	0.4076	22.51	0.7757	0.3848
ReSplat		22.00	0.7594	0.3782	22.90	0.7908	0.3661
DiffUIR	Haze+Snow	15.38	0.6978	0.3488	15.52	0.7702	0.2843
ReSplat		20.17	0.7730	0.3148	19.92	0.8067	0.2808

가장 강력한 baseline인 DiffUIR과 비교 → 전 지표에서 ReSplat 우세

Snow+Motion Blur와 Haze+Snow 같은 더 어려운 scenario에서 높은 복원력 보임

Quantitative Analysis: Real-World Degradation

Method	Type	Novel View Synthesis		
		PSNR(↑)	SSIM(↑)	LPIPS(↓)
AiRnet	Motion Blur	18.59	0.6429	0.4009
PromptIR		18.42	0.6289	0.3959
GAURA		21.54	0.7711	0.3909
DiffUIR		22.76	0.8090	0.2988
ReSplat		22.91	0.8145	0.2922
AiRnet	Haze	15.91	0.7189	0.3290
PromptIR		15.32	0.7106	0.3224
GAURA		16.90	0.8397	0.3920
DiffUIR		17.26	0.8451	0.1900
ReSplat		17.75	0.8511	0.1968
AiRnet	Low-light	9.526	0.1364	0.6040
PromptIR		17.10	0.8322	0.5091
GAURA		19.07	0.8503	0.6301
DiffUIR		22.00	0.8165	0.4958
ReSplat		22.92	0.8578	0.4759

Motion Blur, Haze, Low-light

PSNR, SSIM에서 모두 최고 성능임

※ 표에서 LPIPS가 DiffUIR이 더 낮은데, 본문에서는 LPIPS가 lowest라서 PSNR이랑 SSIM이 DiffUIR이랑 비슷한 수준이지만 perceptual quality가 우수하다고 했음...

Method	Type	PSNR(↑)	SSIM(↑)	LPIPS(↓)
AiRnet	Rain	24.05	0.8183	0.1955
PromptIR		24.23	0.8230	0.1801
GAURA		19.39	0.6602	0.3987
DiffUIR		23.99	0.8145	0.2094
ReSplat		24.35	0.8232	0.1772

Method	Type	PSNR(↑)	SSIM(↑)	LPIPS(↓)
AiRnet	Snow	20.23	0.7035	0.3103
PromptIR		21.27	0.7192	0.3007
GAURA		20.22	0.7578	0.3647
DiffUIR		22.12	0.8215	0.2277
ReSplat		22.45	0.8263	0.2175

Rain

PromptIR 대비 PSNR +0.12 dB이며, 세 지표 모두 최고 성능임

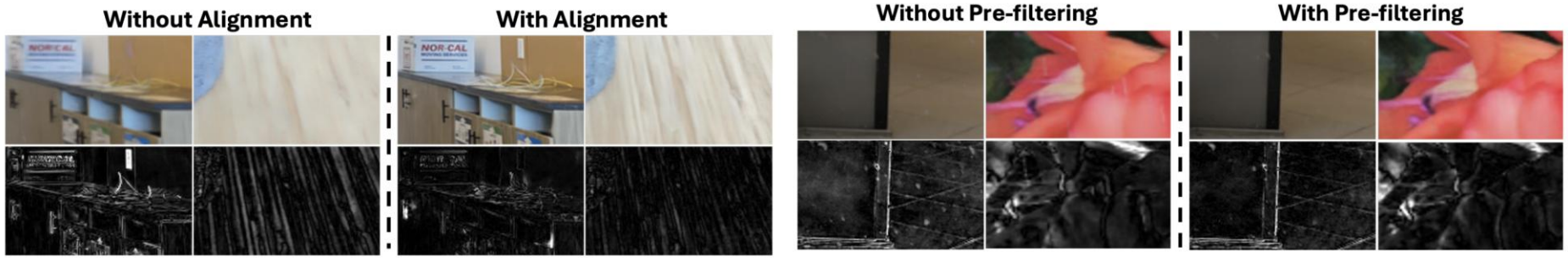
Snow

DiffUIR 대비 PSNR +0.33 dB이며, 세 지표 모두 최고 성능임

Ablation Studies

Model #	Alignment	Pre-Filtering	Novel View Synthesis		
			PSNR(↑)	SSIM(↑)	LPIPS(↓)
1	X	X	22.19	0.8264	0.2372
2	X	O	22.35	0.8290	0.2368
3	O	X	22.46	0.8313	0.2306
4	O	O	22.69	0.8383	0.2230

- Baseline(1) → Pre-filtering만 적용(2) / Alignment만 적용(3): 전 지표 개선
- 두 모듈 **모두 적용**(4): PSNR 22.69, SSIM 0.8383, LPIPS 0.2230으로 **최고 성능!**



Alignment가 multi-view geometric inconsistency를 줄이고,

Pre-filtering이 novel view에서 fine structure를 해치지 않으면서 residual artifact를 억제함

Limitations & Conclusion

Limitations

Diffusion refinement

- Computational & memory overhead
- High-resolution scaling을 위한 추가 optmzt. 필요

3DGS의 specularity, transparency 처리 한계 상속

- Pretrained restoration prior에 의존
- (👍) △ modularity 덕분에 architecture 변경 없이 향후 restoration 발전의 이점 얻을 수 있음

Conclusion

Residual diffusion + 3D cross-attention + multi-view pre-filtering 결합한,
degradation-agnostic NVS를 위한 feed-forward Gaussian Splatting framework 제안

Idea & Discussion

3-view 입력 채택 근거 부족

Supplementary에서 2-view, 4-view에 대한 실험 수행함 ▶ View 수 높아질수록 NVS 성능 개선 경향 발견

Supplementary에 3-view 입력에 대한 NVS inference time 측정한 결과도 있는데, 4-view에 대해서는 X

View 수에 따라 달라지는 inference time도 보여주고 3-view 입력을 정당화했으면 좋았겠다!

Real-world Experiment에서 Multi-view Image Restoration 성능 보고 X

GPT: Input-view restoration을 정량 평가하려면 degraded input과 정확히 대응하는 clean GT가 필요하므로, GT의 확보·정렬 조건이 영향을 줬을 가능성

Thank You